САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 2: взаимодействие с внешним API

Выполнил:

Якшин Артемий

K3440

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Привязать сайт, свёрстанный в ЛР1, к внешнему API средствами fetch, axios или XHR. Реализовать моковое API средствами JSON-сервера и подключить к нему авторизацию, как в примерах с лекций «Имитация работы с API».

Вариант остаётся прежним: сервис для обмена рецептами «Поварёшка». После этой работы все данные сайта должны приходить с сервера: рецепты с фильтрами, комментарии, лайки, сохранённые рецепты, подписки на кулинаров и данные личного кабинета.

Ход работы

1. Моковое API на json-server

Работа начата с копии вёрстки ЛР1. Как и в примере с лекции, API поднимается пакетом `json-server` версии 0.17, а авторизацию добавляет промежуточный слой `json-server-auth`. Флаг `--static .` заставляет тот же сервер раздавать HTML-страницы, поэтому сайт и API работают на одном адресе `http://localhost:3000`.

```
{
  "scripts": {
    "start": "json-server db.json -m ./node_modules/json-server-auth -r routes.json --static . --port 3000"
  },
  "devDependencies": {
    "json-server": "^0.17.4",
    "json-server-auth": "^2.1.0"
  }
}
```

Данные лежат в `db.json`. Связи между коллекциями построены на полях `userId` и `recipeId`, поэтому json-server умеет подтягивать связанные записи параметрами `_expand` и `_embed`.

Таблица 1. Коллекции в db.json

Коллекция	Поля	Записей
`users`	email, password (bcrypt-хеш), name, city, bio, avatarColor	7
`recipes`	userId, title, description, image, type, difficulty, time, servings, ingredients[], tags[], steps[], video, createdAt	11
`comments`	recipeId, userId, text, createdAt	9
`likes`	userId, recipeId	37
`saves`	userId, recipeId	6
`subscriptions`	userId (подписчик), authorId (кулинар)	8

Пароли хранятся в виде bcrypt-хешей, как этого требует json-server-auth. У всех демо-пользователей пароль `povar2026`, для входа можно использовать аккаунт `demo@povaryoshka.ru`.

Права доступа заданы в `routes.json`. Три цифры означают права владельца, авторизованного пользователя и гостя (4 — чтение, 2 — запись). Например, 644 разрешает читать всем, а менять только автору записи:

```
{
  "/users*": "/644/users$1",
  "/recipes*": "/644/recipes$1",
  "/comments*": "/644/comments$1",
  "/likes*": "/644/likes$1",
  "/subscriptions*": "/644/subscriptions$1",
  "/saves*": "/600/saves$1"
}
```

Правила проверены запросами через `curl` под демо-пользователем (id 5). Результаты в таблице ниже.

Таблица 2. Проверка прав доступа

Запрос	**Ответ**
`POST /likes` без токена	401 Missing authorization header
`POST /likes` с `"userId": 1` (от имени другого пользователя)	403 Private resource creation: request body must have a reference to the owner id
`DELETE /recipes/1` (рецепт Анны)	403 Private resource access: entity must have a reference to the owner id
`GET /saves?userId=5` без токена	401 Missing authorization header

> При проверке выяснилась особенность json-server-auth: для списков под правилом 600 он не смотрит на параметры запроса и только проверяет, есть ли в коллекции хотя бы одна запись текущего пользователя (в исходниках модуля на этом месте стоит TODO). Поэтому у пользователя без сохранений вместо пустого массива приходит 403, а пользователь с сохранениями технически может прочитать чужой список через `?userId=`. Для учебного мока это допустимо: клиент всегда запрашивает только свой `userId`, а ответ 403 на пустой список считает отсутствием сохранений.

2. Клиент API

Все запросы собраны в модуле `js/api.js`. Скрипты подключаются как ES-модули (`<script type="module">`), поэтому каждая страница импортирует только нужные функции. Основная функция `request()` собирает URL с параметрами, добавляет заголовок `Authorization` с токеном и разбирает ответ:

```
async function request(path, { method = 'GET', body, params, auth = false } = {}) {
  const url = new URL(path, API_URL)

  // { type: ['Супы', 'Завтраки'] } превращается в ?type=Супы&type=Завтраки
  Object.entries(params ?? {}).forEach(([key, value]) => {
    ;[].concat(value).forEach((item) => {
      if (item !== undefined && item !== null && item !== '')
url.searchParams.append(key, item)
    })
  })

  const headers = {}
  if (body) headers['Content-Type'] = 'application/json'
  if (auth) headers.Authorization = `Bearer ${getToken()}`

  const response = await fetch(url, { method, headers, body: body ?
JSON.stringify(body) : undefined })
  const data = await response.json().catch(() => null)

  if (!response.ok) {
    // токен json-server-auth живёт час: после этого сессию нужно начать заново
    if (response.status === 401 && auth) {
      logout()
      throw new ApiError(401, 'Сессия истекла, войдите ещё раз')
    }
    throw new ApiError(response.status, typeof data === 'string' ? data : 'Ошибка
запроса')
  }
  return data
}

export const getRecipes = (params = {}) =>
  request('/recipes', { params: { _expand: 'user', _embed: 'likes', ...params } })

export const addLike = (userId, recipeId) =>
  request('/likes', { method: 'POST', body: { userId, recipeId }, auth: true })
```

Кроме `api.js`, есть ещё четыре модуля:

- `auth.js` хранит токен и пользователя в `localStorage` и умеет отправить гостя на страницу входа;
- `ui.js` содержит общие шаблоны: карточку рецепта, аватар, шапку и уведомления;
- `saves.js` обслуживает кнопку-закладку на карточках;
- `pages/*.js` содержат по скрипту на каждую страницу.

Данные, введённые пользователями (названия, комментарии, имена), вставляются в разметку только через функцию `escapeHtml()`, чтобы нельзя было внедрить чужой HTML или скрипт.

3. Регистрация и вход

Форма входа отправляет `POST /login`. В ответ json-server-auth возвращает JWT-токен и данные пользователя, которые сохраняются в `localStorage`. Ошибки сервера переводятся

на русский и показываются над формой (рис. 1). Регистрация устроена так же, через `POST /register`. При повторной регистрации на ту же почту сервер отвечает «Email already exists» (рис. 2).

```
try {
  const session = await login(form.email.value.trim(), form.password.value)
  saveSession(session)          // localStorage.accessToken и localStorage.user
  location.href = nextPage()    // туда, откуда пришли (?next=...), или в кабинет
} catch (error) {
  errorBox.textContent = messages[error.message] ?? error.message
  errorBox.classList.remove('d-none')
}
```

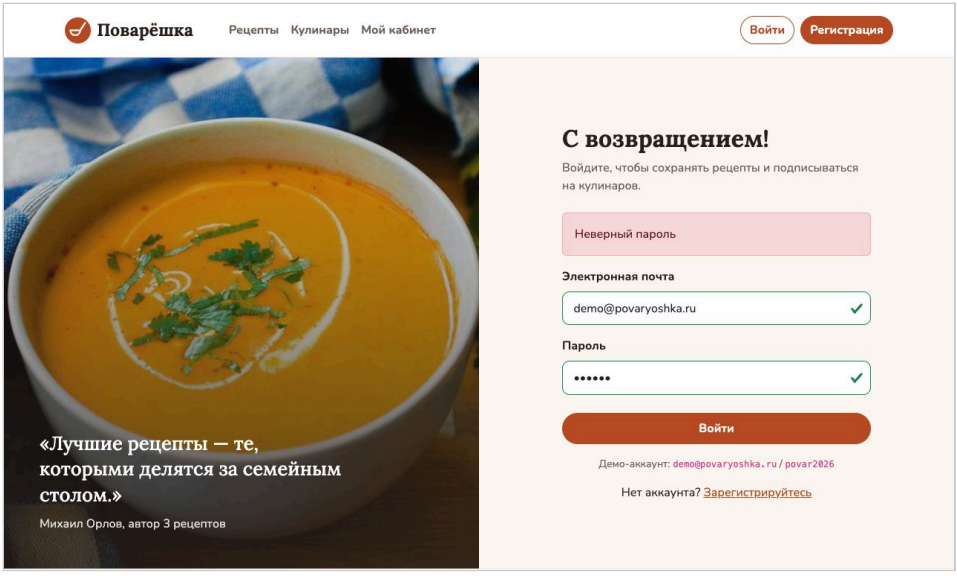


Рисунок 1. Вход с неверным паролем: ошибка от API

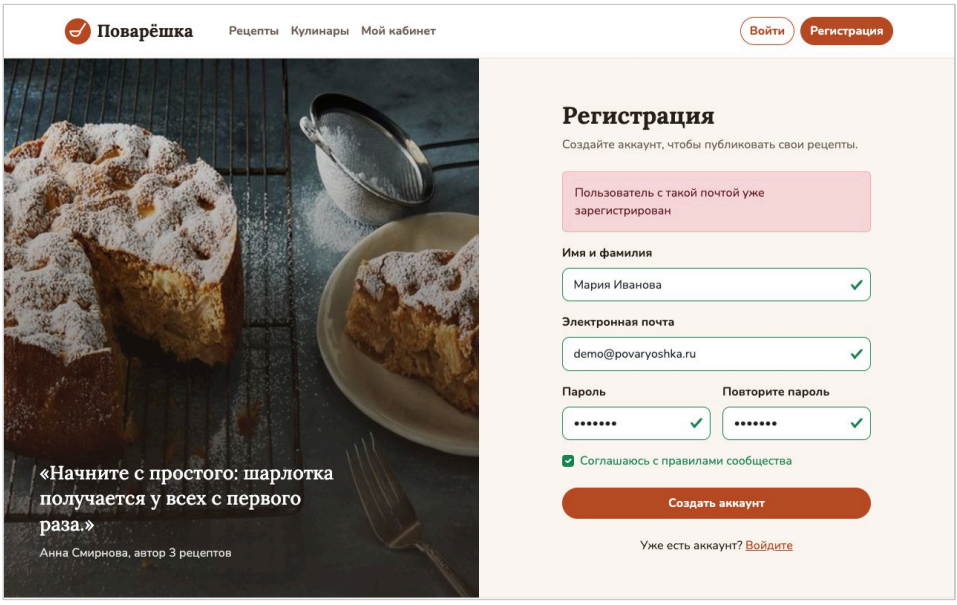


Рисунок 2. Регистрация на уже занятую почту

После входа шапка перерисовывается: вместо кнопок «Войти» и «Регистрация» появляется меню пользователя со ссылками на кабинет, свою страницу и кнопкой выхода (рис. 3). Личный кабинет защищён: гостя функция `requireAuth()` отправляет на `login.html?next=/profile.html`, а после входа он возвращается обратно.

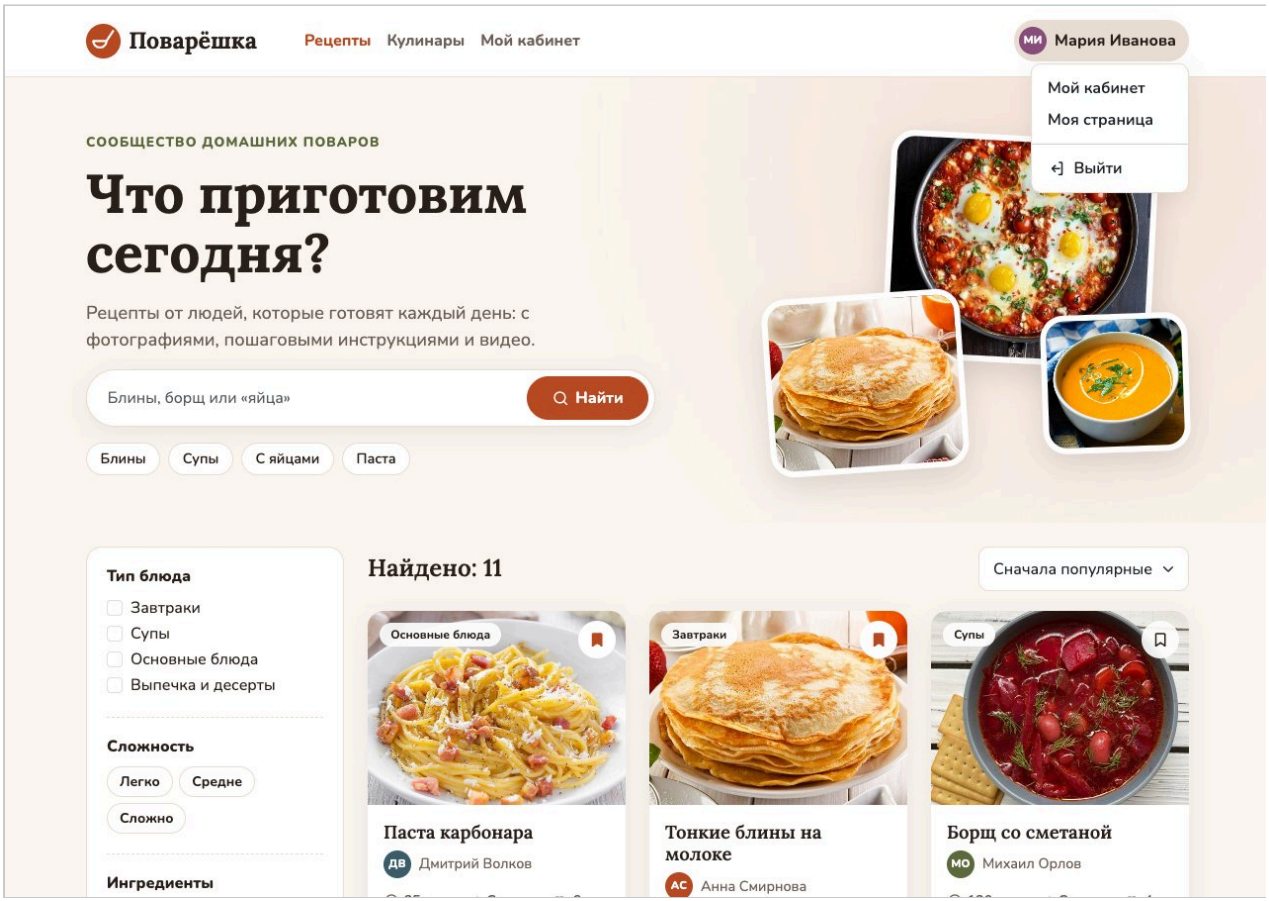


Рисунок 3. Главная страница после входа, открыто меню пользователя

4. Поиск рецептов с фильтрацией через API

Фильтры главной страницы превращаются в параметры запроса к json-server. Повторяющийся параметр работает как «или» (`type=Супы&type=Завтраки`), `time_lte` ограничивает время, `q` ищет по всем полям рецепта, включая ингредиенты, а `_sort` и `_order` сортируют. Условие «рецепт содержит все выбранные ингредиенты» json-server выразить не может, поэтому этот фильтр применяется к ответу уже на клиенте. Так же считается сортировка по популярности: число лайков приходит благодаря `_embed=likes`.

```
const params = { type: filters.type, difficulty: filters.difficulty, time_lte:
filters.time, q: filters.q }
if (filters.sort === 'date') Object.assign(params, { _sort: 'createdAt', _order:
'desc' })
if (filters.sort === 'time') Object.assign(params, { _sort: 'time', _order: 'asc' })

let recipes = await getRecipes(params)
// пока ждали ответ, пользователь мог поменять фильтры: старый ответ не рисуем
if (requestId !== lastRequest) return
```

```
recipes = recipes.filter((recipe) => filters.tags.every((tag) =>
recipe.tags.includes(tag)))
if (filters.sort === 'likes') recipes.sort((a, b) => b.likes.length - a.likes.length)
```

Например, фильтр «Супы» + «Легко» отправляет запрос `/recipes?_expand=user&_embed=likes&type=Супы&difficulty=Легко`. В ответе приходит рецепт вместе с автором и лайками:

```
[{
  "id": 6,
  "title": "Тыквенный крем-суп",
  "type": "Супы",
  "difficulty": "Легко",
  "time": 40,
  "userId": 2,
  "user": { "id": 2, "name": "Михаил Орлов", ... },
  "likes": [{ "id": 22, "userId": 3, "recipeId": 6 }, { "id": 23, "userId": 5,
"recipeId": 6 }]
}]
```

Поиск по строке срабатывает с паузой 300 мс после последнего нажатия клавиши, чтобы не отправлять запрос на каждую букву. Номер последнего запроса защищает от гонки: если пользователь быстро переключает фильтры, на экран попадает ответ только на последний из них (рис. 4).

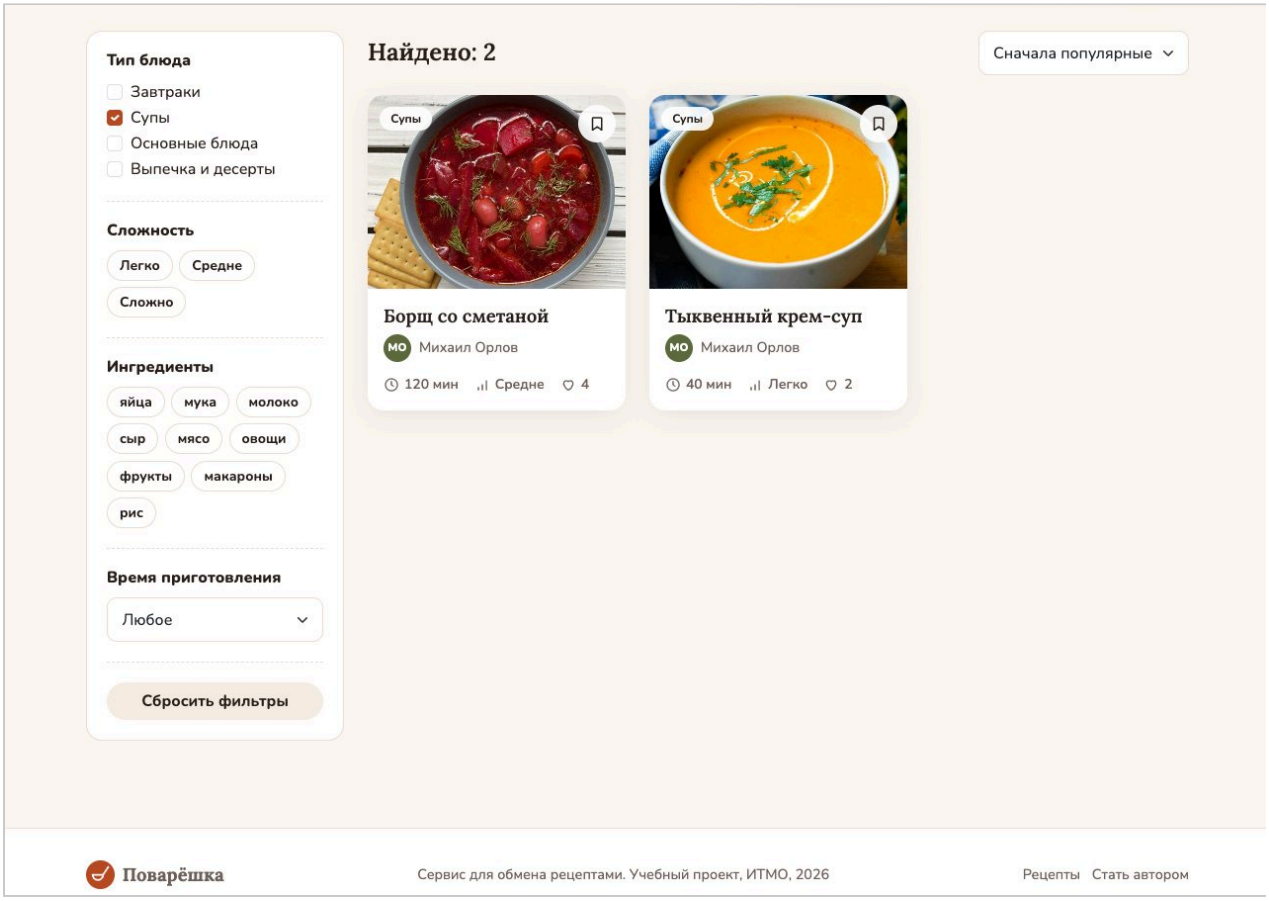


Рисунок 4. Фильтр «Супы»: два рецепта из API

5. Страница рецепта: лайки, сохранения, комментарии

Страница открывается по адресу `recipe.html?id=1`. Рецепт, комментарии и лайки загружаются параллельно через `Promise.all`, затем подгружаются число рецептов и подписчиков автора, а для вошедшего пользователя ещё и его сохранения:

```
const [recipe, comments, likes] = await Promise.all([
  api.getRecipe(recipeId),      // /recipes/1?_expand=user
  api.getComments(recipeId),    // /comments?
recipeId=1&_expand=user&_sort=createdAt&_order=desc
  api.getLikes({ recipeId })    // /likes?recipeId=1
])
```

Кнопка «Нравится» создаёт запись `POST /likes` или удаляет её `DELETE /likes/:id`. Так же работают «Сохранить» (`/saves`) и «Подписаться» (`/subscriptions`). Если гость нажимает такую кнопку, его отправляют на страницу входа. Новый комментарий отправляется запросом `POST /comments` и сразу появляется в начале списка. Свой комментарий можно удалить (рис. 5, 6).

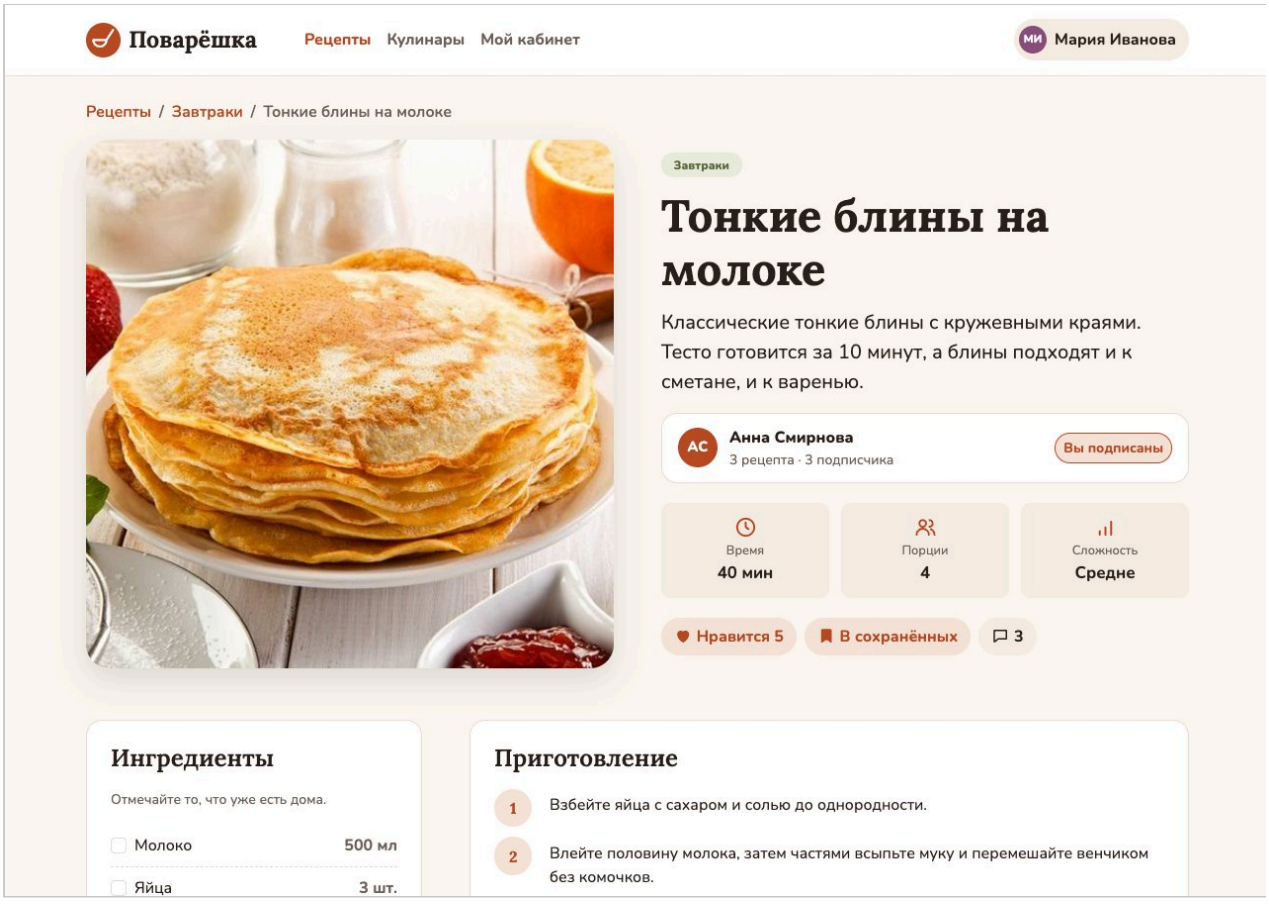


Рисунок 5. Страница рецепта: лайк, сохранение и подписка уже отмечены для текущего пользователя

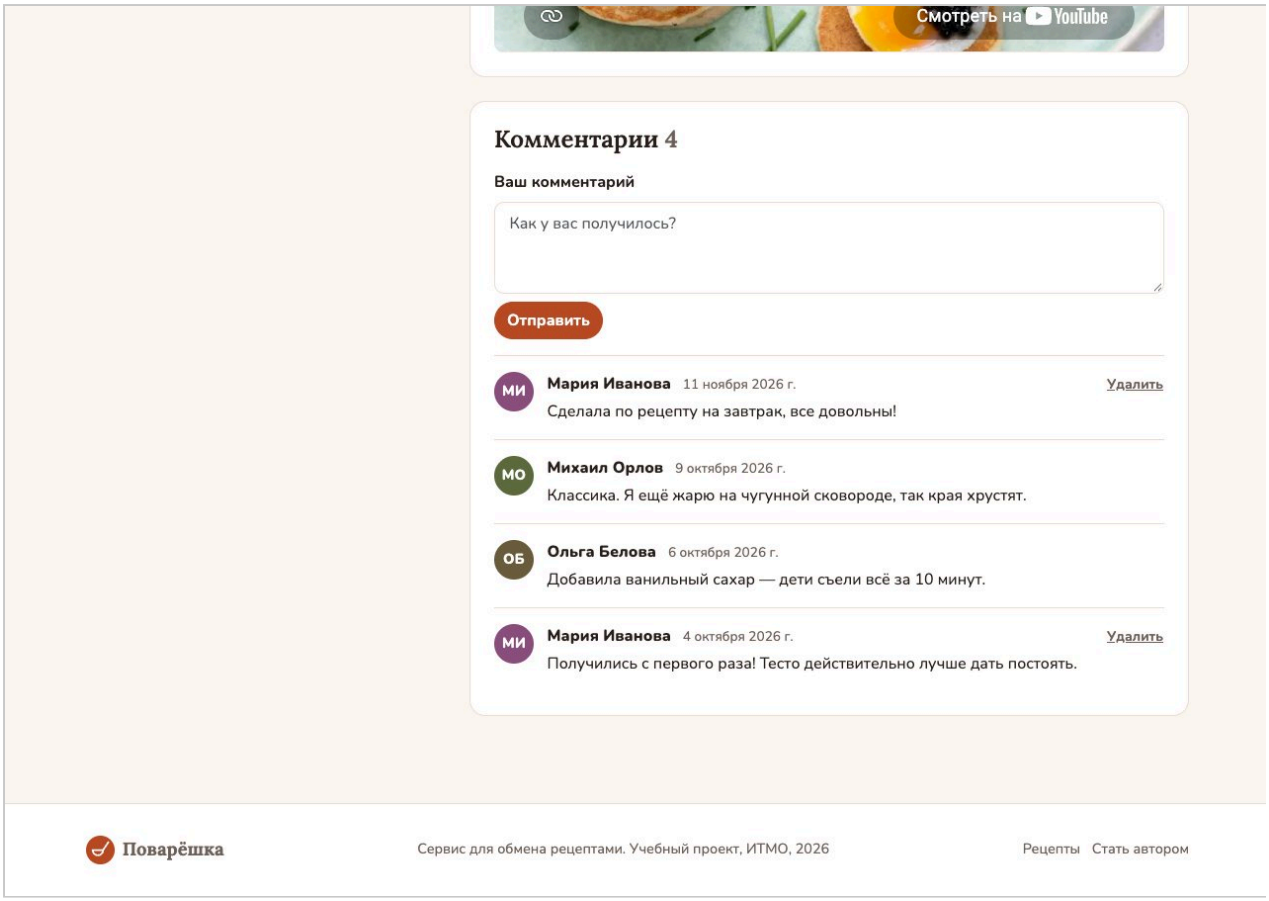


Рисунок 6. Добавленный комментарий с кнопкой «Удалить»

6. Кулинары и подписки

Страница `author.html` без параметра показывает список кулинаров (рис. 7). Число рецептов и подписчиков считается по ответам `/recipes` и `/subscriptions`. С параметром `?id=1` открывается страница одного автора с кнопкой подписки и его рецептами (рис. 8).

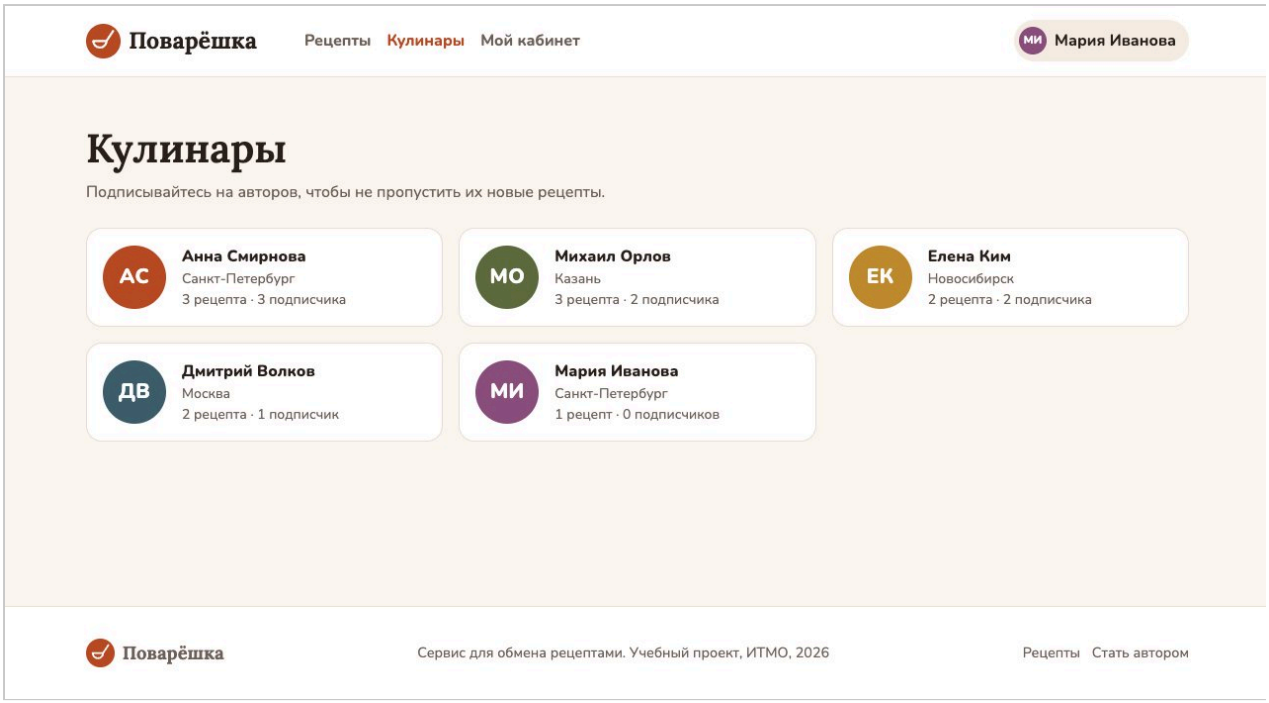


Рисунок 7. Список кулинаров

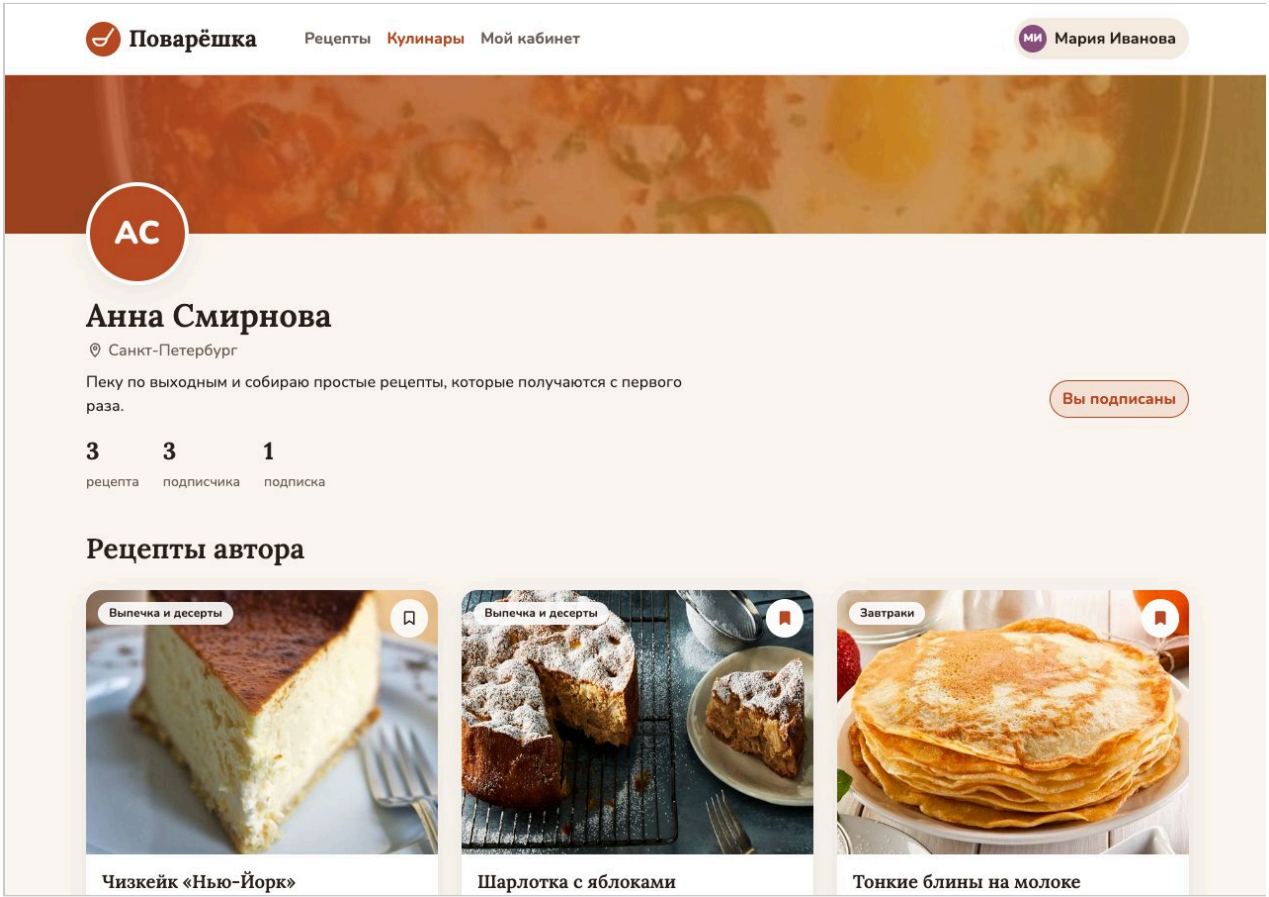


Рисунок 8. Страница кулинара: текущий пользователь уже подписан

7. Личный кабинет

Кабинет (рис. 9) собирает данные из нескольких запросов: профиль `/users/5`, свои рецепты `/recipes?userId=5`, сохранения `/saves?userId=5` и подписки `/subscriptions?userId=5`. Сохранённые рецепты и авторов, на которых подписан пользователь, удобно получить одним запросом с повторяющимся параметром `id`, например `/recipes?id=1&id=3&id=4&id=7`.

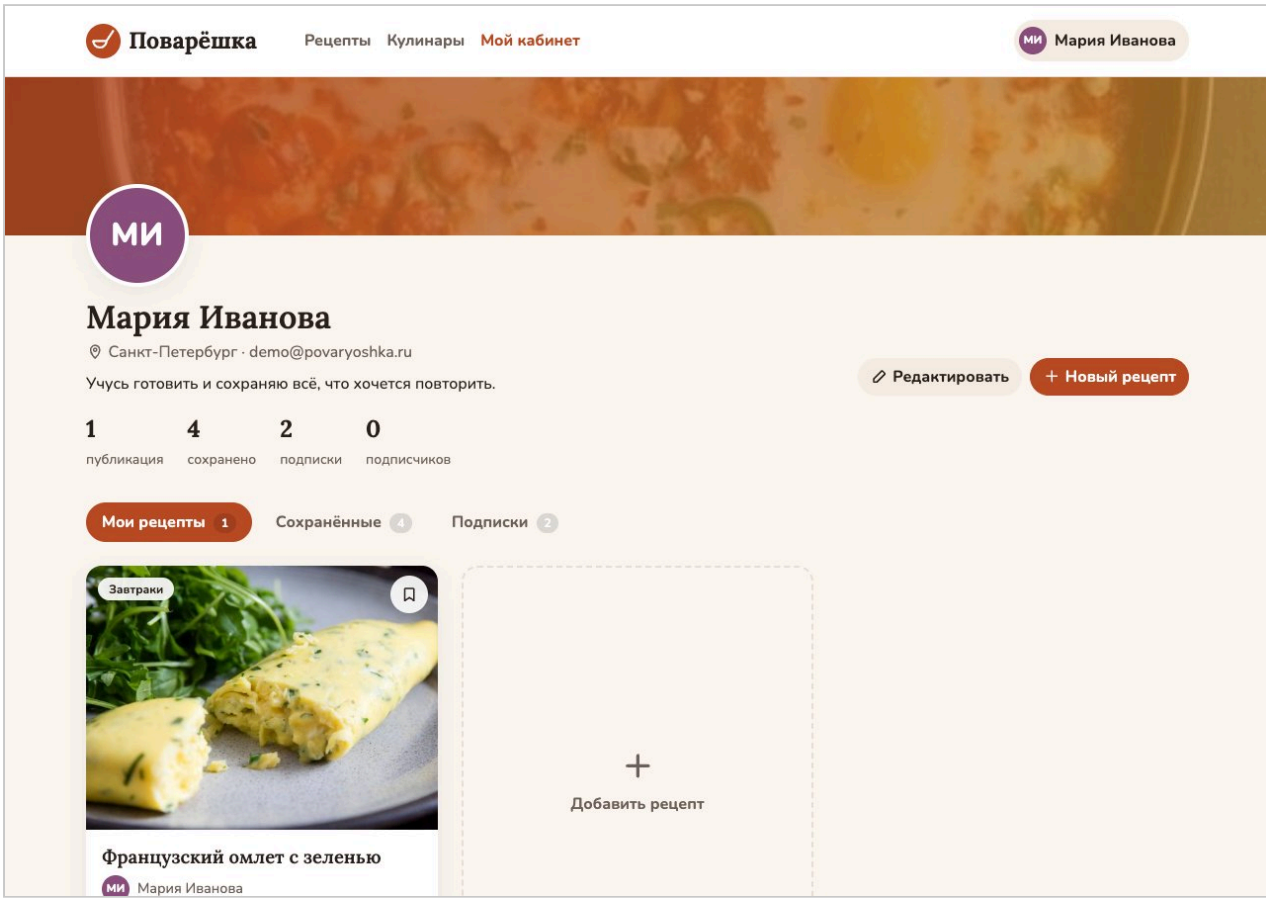


Рисунок 9. Личный кабинет с данными из API

Модальное окно из ЛР1 теперь создаёт рецепт (`POST /recipes`) или изменяет существующий (`PATCH /recipes/:id`). Ингредиенты вводятся строками вида «Творог — 400 г» и превращаются в массив объектов. Теги для фильтра «Ингредиенты» подбираются автоматически по названиям продуктов, а из ссылки на YouTube извлекается идентификатор видео (рис. 10, 11).

```
const TAG_RULES = {
  яйца: /яйц|желтк|белк/i,
  мука: /мук/i,
  молоко: /молок|сливк|сметан|кефир/i,
  ...
}

function detectTags(ingredients) {
  return Object.keys(TAG_RULES).filter((tag) =>
```

```
    ingredients.some((item) => TAG_RULES[tag].test(item.name)))
}
```

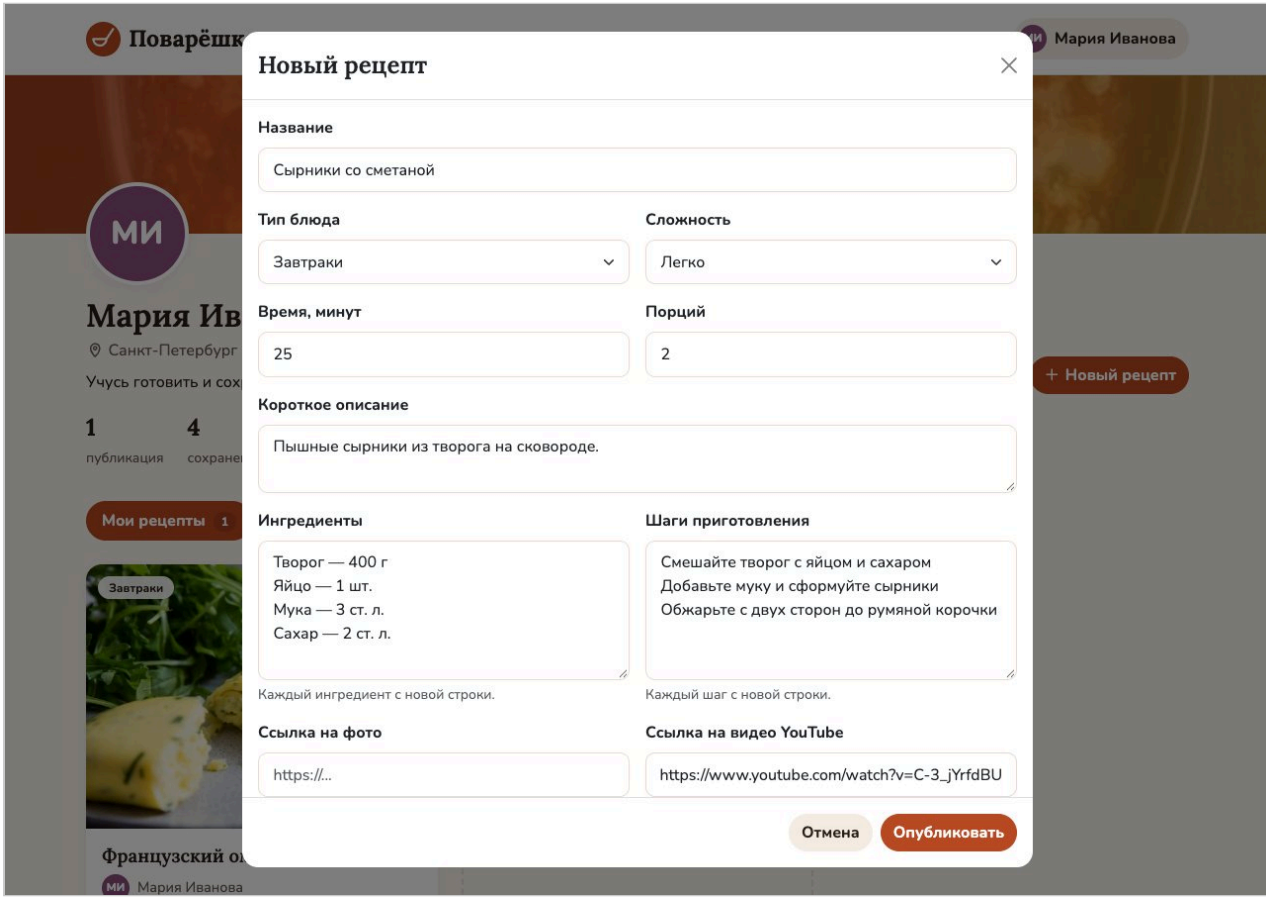


Рисунок 10. Заполненная форма нового рецепта

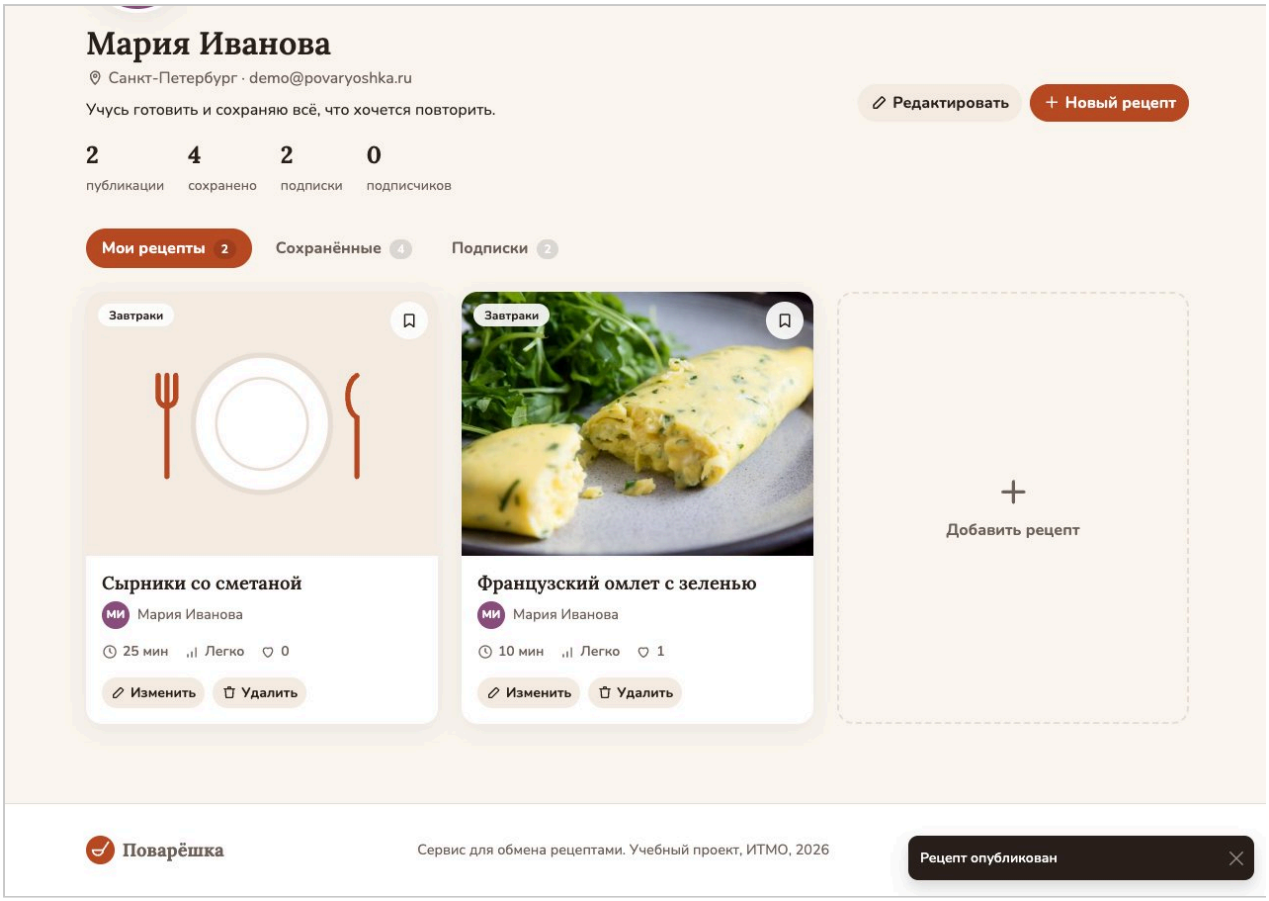


Рисунок 11. Рецепт опубликован: карточка с заглушкой вместо фото и уведомление

Удаление рецепта (`DELETE /recipes/:id`) подтверждается в модальном окне. json-server сам удаляет записи, которые ссылаются на рецепт через `recipeId`: лайки, комментарии и сохранения. Это проверено запросом `/likes?recipeId=…` после удаления, он вернул пустой массив. Данные профиля меняются запросом `PATCH /users/5`, после чего обновляются шапка и сохранённая сессия.

8. Проверка

Сценарий целиком прогнан автоматически в браузере против запущенного json-server. Проверены:

- фильтры и сортировка;
- вход с неверным и верным паролем;
- создание, изменение и удаление рецепта;
- лайк, комментарий и его удаление;
- подписка и сохранение;
- редактирование профиля;
- выход и защита кабинета;
- регистрация нового пользователя.

Во время прогона была найдена и исправлена ошибка: у нового пользователя без сохранений кабинет не загружался из-за ответа 403 на пустой список `/saves` (см. раздел 1).

Вывод

Сайт из ЛР1 переведён на работу с моковым REST API. json-server хранит данные и поддерживает фильтрацию, сортировку и связи между коллекциями, json-server-auth выдаёт JWT-токены и ограничивает права через `routes.json`. Запросы на клиенте выполняются через `fetch` и собраны в одном модуле, поэтому страницы не повторяют код работы с сетью и токеном. Все пункты варианта работают с настоящими данными:

- поиск с фильтрами;
- страница рецепта с видео;
- лайки, комментарии и подписки;
- личный кабинет с публикациями и сохранёнными рецептами.